



Atividade em Grupo

Atividade - Aplicando Técnicas de DevOps

Período: de 10/03/2026 a 21/04/2026

Integrantes:

Ana Clara da Silva Reis	RM560789
Cyntia Luiza Hagers	RM560525

EcoPulse – Plataforma ESG para Cidades Inteligentes

Aplicação de Técnicas DevOps

1. Introdução e Justificativa da Solução

Para esta atividade foi escolhida a abordagem de inovação, desenvolvendo uma nova aplicação chamada EcoPulse, projetada desde o início para utilizar o banco de dados NoSQL (MongoDB).

A decisão por inovar permitiu explorar integralmente as características do MongoDB, especialmente: flexibilidade de esquema orientado a documentos, escalabilidade horizontal, suporte a grandes volumes de dados IoT e armazenamento eficiente de estruturas heterogêneas e aninhadas.

A plataforma EcoPulse foi desenvolvida utilizando MongoDB devido às características técnicas necessárias para aplicações ESG em cidades inteligentes.

Soluções ESG envolvem coleta contínua de dados de sensores IoT, com estruturas variáveis e alto volume de registros. O MongoDB é adequado nesse cenário por permitir:

- **Esquema flexível**, possibilitando armazenar diferentes tipos de medições (energia, qualidade do ar, resíduos) na mesma collection;
- **Escalabilidade horizontal**, suportando crescimento do volume de dados;
- **Armazenamento de documentos aninhados**, facilitando registros de auditoria e conformidade;
- **Alta performance para leitura e escrita**, essencial para dados em tempo real.

Na collection readings, por exemplo, cada documento pode conter campos diferentes conforme o tipo de sensor, demonstrando a principal vantagem do modelo NoSQL em comparação aos bancos relacionais tradicionais. Essa característica reduz a necessidade de remodelagem estrutural do banco, tornando a solução mais adaptável à evolução dos sensores e indicadores ESG.

Assim, a escolha do MongoDB contribui diretamente para resolver desafios ambientais (monitoramento contínuo), sociais (transparência e responsabilização) e de governança (registro de conformidade e auditorias).

Além disso, o projeto foi evoluído com a aplicação de práticas DevOps, incluindo containerização com Docker, orquestração com Docker Compose e automação de



integração contínua com GitHub Actions, simulando um ambiente mais próximo de produção.

2. Arquitetura

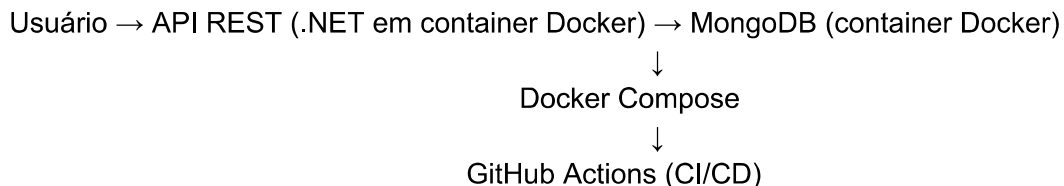
A arquitetura do sistema EcoPulse foi estruturada para simular um ambiente real de produção, integrando a API REST em .NET, o banco de dados MongoDB e práticas DevOps para automação e padronização do ambiente.

A aplicação foi containerizada utilizando Docker, permitindo que tanto a API quanto o banco de dados sejam executados em containers isolados entre si. A orquestração desses serviços é realizada por meio do Docker Compose, que gerencia a comunicação entre os containers, redes e volumes necessários para o funcionamento do sistema.

A comunicação entre a API e o banco de dados ocorre através do nome do serviço definido no Docker Compose, garantindo desacoplamento e facilidade de configuração.

Além disso, foi implementado um pipeline de integração contínua utilizando GitHub Actions, responsável por automatizar etapas como build da aplicação, execução de testes, build da imagem Docker e simulação de deploy automatizado nos ambientes de staging e produção.

Dessa forma, a arquitetura do sistema pode ser representada da seguinte forma:



3. Banco de Dados (MongoDB)

O banco de dados utilizado na aplicação é o MongoDB, executado em container Docker e orquestrado por meio do Docker Compose, garantindo isolamento, portabilidade e facilidade de configuração do ambiente.

A modelagem adotou referências entre collections por meio de identificadores como `device_id`, `reading_id` e `owner_user_id`, permitindo associação lógica entre dispositivos, leituras, alertas e responsáveis. Essa estratégia mantém o desacoplamento estrutural



característico de bancos NoSQL, preservando a flexibilidade e escalabilidade da aplicação.

Além disso, foi utilizado volume no container do MongoDB para garantir persistência dos dados, mesmo após reinicialização do ambiente.

Collection: users

Representa o aspecto Social e de Governança, armazenando usuários responsáveis pela gestão da plataforma.

Collection: devices

Representa o aspecto Ambiental, armazenando sensores IoT responsáveis pela coleta de dados ambientais.

Collection: readings

Representa o aspecto Ambiental, armazenando medições variáveis coletadas pelos dispositivos. Esses dados servem como base para geração de indicadores e relatórios ESG.

Collection: alerts

Representa os aspectos Ambiental e de Governança, registrando alertas automáticos e controle de incidentes. Permite rastreabilidade e acompanhamento de ações corretivas.

Collection: compliance_records

Representa o aspecto de Governança, armazenando registros de auditorias, normas e metas ESG. Garante controle de prazos e conformidade regulatória.

As collections são criadas dinamicamente pela aplicação no momento da inserção de dados, conforme o comportamento padrão do MongoDB. Dessa forma, não há necessidade de criação manual prévia das estruturas, garantindo maior flexibilidade e aderência ao modelo NoSQL.

4. Criação das Collections

No contexto desta aplicação, as collections do MongoDB não são criadas manualmente via comandos, como ocorre em abordagens tradicionais utilizando MongoDB Compass ou mongosh.

Como a aplicação está integrada ao banco de dados por meio de uma API em .NET, executada em ambiente containerizado com Docker, as collections são criadas automaticamente no momento da inserção dos primeiros dados.

Essa abordagem segue o comportamento padrão do MongoDB, no qual o banco de dados e suas collections são gerados dinamicamente conforme a aplicação realiza operações de escrita.

Dessa forma, ao realizar requisições do tipo POST na API (por exemplo, cadastro de dispositivos), o MongoDB cria automaticamente a collection correspondente, caso ela ainda não exista.

Esse modelo reforça as características do NoSQL, como flexibilidade de esquema e menor acoplamento estrutural, além de estar alinhado com práticas modernas de desenvolvimento e DevOps.

5. Operações CRUD Realizadas

Operações CRUD Realizadas via API

Nesta aplicação, as operações de CRUD (Create, Read, Update e Delete) são realizadas por meio da API REST desenvolvida em .NET, utilizando o Swagger como interface para testes e validações.

Diferentemente de abordagens tradicionais que utilizam comandos diretos no banco de dados (mongosh ou MongoDB Compass), nesta solução todas as interações com o MongoDB são feitas por meio da aplicação, reforçando o desacoplamento e boas práticas de arquitetura.

Create (POST)

A criação de registros é realizada através do endpoint:

POST /devices

Exemplo de requisição:

```
{
  "type": "Sensor",
  "model": "ESP32"
}
```

Ao executar essa operação, a API insere o documento no MongoDB, criando automaticamente a coleção caso ainda não exista.

Figura X – Criação de dispositivo via API (POST /devices)

Fonte: Elaborado pela autora (2026).

Read (GET)

A consulta de dados é realizada através do endpoint:

GET /devices

Essa operação retorna todos os dispositivos cadastrados no banco de dados, evidenciando a persistência das informações.

Figura X – Consulta de dispositivos via API (GET /devices)

Fonte: Elaborado pela autora (2026).

Observação

As operações são executadas via API, e não diretamente no banco de dados, garantindo maior segurança, organização e alinhamento com arquiteturas modernas baseadas em serviços.

6. API

Foi desenvolvida uma API REST utilizando .NET 8 para permitir a integração com o banco de dados MongoDB e o acesso aos dados ESG da aplicação EcoPulse.

A API foi estruturada seguindo boas práticas de desenvolvimento em camadas, incluindo Controllers, Services e Models, garantindo organização, manutenção e escalabilidade do código.

Os endpoints disponibilizados permitem operações básicas sobre os dados, como criação e consulta de dispositivos.

Principais endpoints:

- GET /devices → retorna os dispositivos cadastrados
- POST /devices → cria um novo dispositivo

A API foi testada utilizando o Swagger, permitindo validar as requisições e respostas de forma interativa. As evidências de execução da API podem ser visualizadas na seção de Evidências.

7. Containerização com Docker

A aplicação foi containerizada utilizando Docker, permitindo a padronização do ambiente de execução e facilitando a replicação em diferentes máquinas.

Foi utilizado um Dockerfile para criação da imagem da aplicação .NET, contendo todas as dependências necessárias para execução.

Além disso, foi utilizado o Docker Compose para orquestrar múltiplos serviços, permitindo a execução integrada da API e do banco de dados MongoDB em containers separados.

A comunicação entre os serviços ocorre por meio de uma rede interna definida no Docker Compose, garantindo integração entre a aplicação e o banco de dados.

Também foi configurado um volume para o container do MongoDB, garantindo a persistência dos dados mesmo após a reinicialização dos containers.

Essa abordagem proporciona maior portabilidade, isolamento e escalabilidade da aplicação, além de estar alinhada com práticas modernas de DevOps.

As evidências da execução em ambiente containerizado estão disponíveis na seção de Evidências.

8. Pipeline CI/CD

Foi implementado um pipeline de integração contínua e entrega contínua (CI/CD) utilizando GitHub Actions. O pipeline foi estruturado para automatizar o ciclo de vida da aplicação, desde a integração do código até a simulação de deploy, garantindo maior confiabilidade, padronização e redução de erros humanos.

O pipeline é executado automaticamente a cada push ou pull request nas branches principais do projeto (main e staging).

As etapas automatizadas incluem:

- Checkout do código-fonte
- Configuração do ambiente .NET 8
- Restauração das dependências
- Build da aplicação



- Execução de testes automatizados
- Build da imagem Docker
- Execução de container para validação
- Simulação de deploy em ambiente de staging e produção

Essa automação garante que qualquer alteração no código seja validada automaticamente, reduzindo falhas e aumentando a qualidade da aplicação.

9. Ambientes de Deploy

Foram definidos dois ambientes para simulação de um fluxo real de entrega contínua:

- Staging: ambiente destinado a testes e validação das alterações antes da liberação final. Esse ambiente é acionado automaticamente quando há alterações na branch "staging".
- Produção: ambiente que representa a versão final da aplicação, utilizada como referência estável. Esse ambiente é acionado automaticamente quando há alterações na branch "main".

A separação entre os ambientes permite maior controle sobre o ciclo de desenvolvimento, garantindo que novas funcionalidades sejam validadas antes de serem consideradas prontas para produção.

Essa abordagem segue boas práticas de DevOps, promovendo organização, segurança e qualidade no processo de entrega da aplicação.

As evidências da execução do pipeline CI/CD podem ser visualizadas na seção de Evidências.

10. Evidências

As evidências abaixo demonstram a execução do banco de dados, inserção de dados e funcionamento da aplicação.

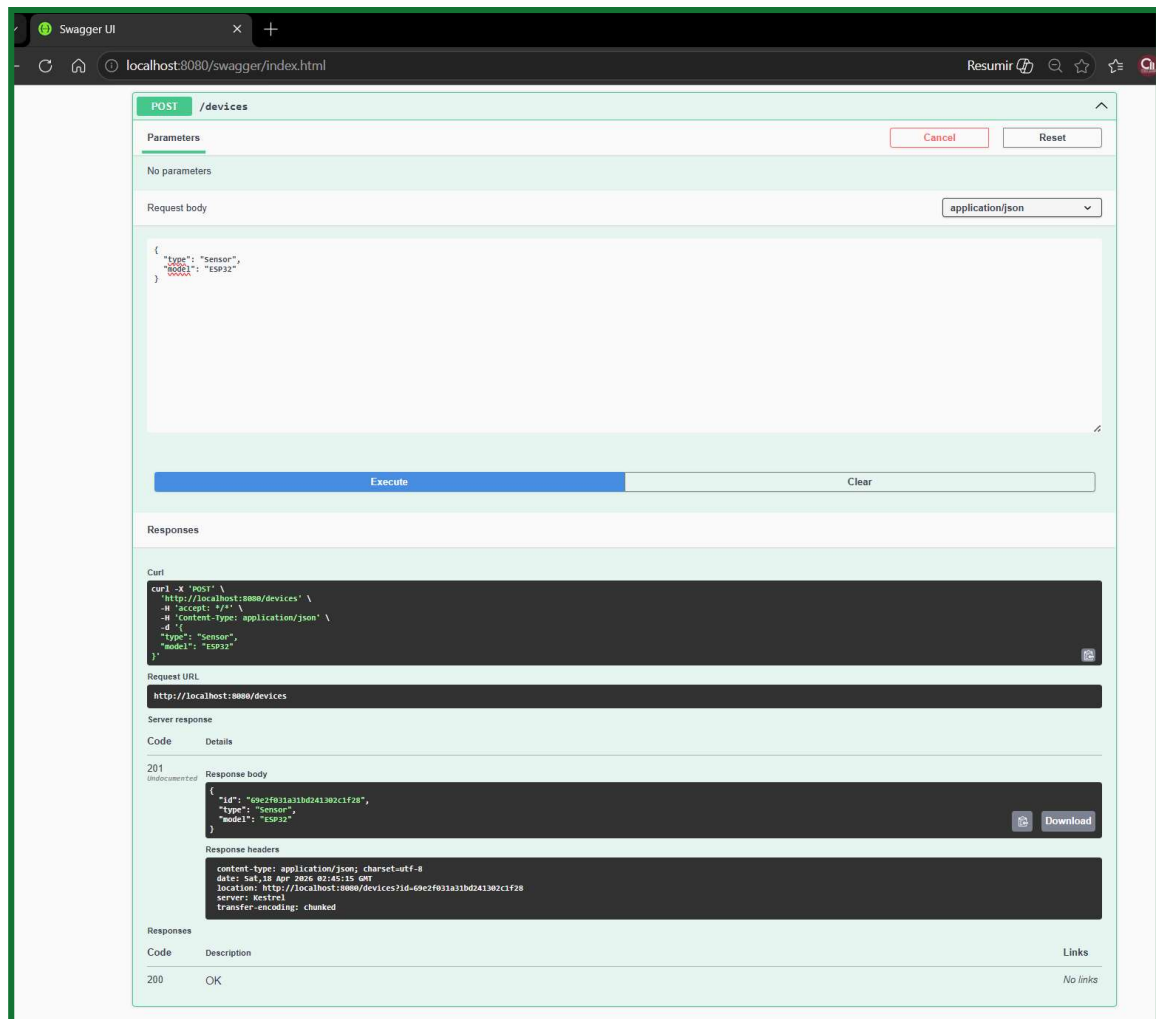


Figura 1 – Execução da API via Swagger (POST)

Fonte: Elaborado pela autora (2026).

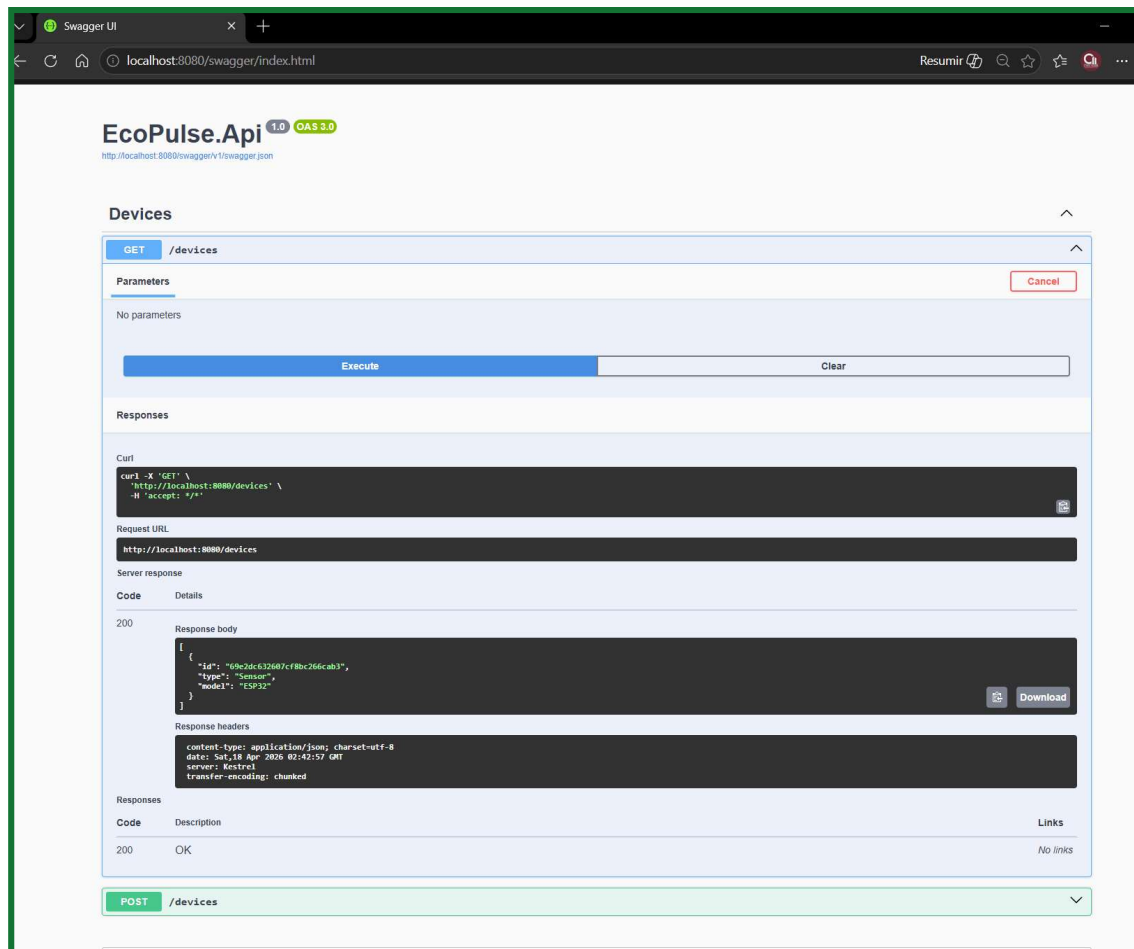
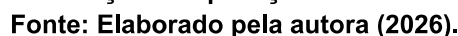


Figura 2 – Execução da API via Swagger (GET)
Fonte: Elaborado pela autora (2026).



11. Desafios Encontrados

Durante o desenvolvimento do projeto, foram identificados alguns desafios técnicos relevantes, principalmente relacionados à evolução de uma solução inicialmente focada apenas em banco de dados para uma arquitetura completa com práticas DevOps.

Dentre os principais desafios, destacam-se:

- Integração de uma API REST a um projeto originalmente baseado apenas em modelagem e manipulação de dados no MongoDB;
- Configuração do ambiente Docker para execução integrada da aplicação e do banco de dados, garantindo comunicação entre os serviços;
- Implementação do pipeline CI/CD com GitHub Actions, incluindo automação de build, testes e preparação para deploy.

Esses desafios foram superados por meio de estudo prático, testes iterativos e adaptação da arquitetura da aplicação, permitindo a evolução do projeto para um cenário mais próximo de um ambiente real de produção.

12. Conclusão

O projeto EcoPulse demonstra que o uso de bancos NoSQL, especialmente o MongoDB, é altamente adequado para aplicações ESG em cidades inteligentes.

A flexibilidade de esquema, aliada à escalabilidade e ao suporte a dados heterogêneos, permite armazenar e analisar informações ambientais de forma eficiente e adaptável, atendendo às demandas de monitoramento contínuo de sensores IoT.

Com a evolução do projeto, foram incorporadas práticas DevOps, incluindo containerização com Docker, orquestração com Docker Compose e implementação de pipeline CI/CD com GitHub Actions.

Essas práticas permitiram simular um ambiente real de produção, garantindo maior automação, padronização, escalabilidade e confiabilidade da aplicação.

Dessa forma, o EcoPulse não apenas demonstra a viabilidade técnica do uso de MongoDB para ESG, mas também sua maturidade para aplicação em cenários reais, integrando desenvolvimento e operações de forma eficiente.

13. Checklist de Entrega

Projeto compactado em .ZIP com estrutura organizada ☒

Dockerfile funcional ☒

docker-compose.yml ou arquivos Kubernetes ☒

Pipeline com etapas de build, teste e deploy ☒

README.md com instruções e prints ☒

Documentação técnica com evidências (PDF ou PPT) ☒

Deploy realizado nos ambientes staging e produção ☒